AGL Lakehouse — Medallion Implementation Guide

Step-by-step Bronze → Silver → Gold build on Databricks, file-only ingestion, and CI/CD to GitHub

This guide implements the full medallion architecture for the 31 tables catalogued in `AGL_Data_Dictionary_Verified.pdf`, across all six source folders. It assumes **no external database** — every table, including the three master files, is uploaded by hand into a Unity Catalog Volume and picked up by Auto Loader. It is built directly on the verified facts from that data dictionary: real row counts, the formats that actually exist per table, and the cross-domain key mismatches found in its Part 8 — this guide shows you exactly where those mismatches bite and how to handle them in Silver, instead of pretending they don't exist.

Setting	Value used in this guide
Catalog	`workspace`
Landing schema (hosts the Volume)	`workspace.landing`
Volume	`/Volumes/workspace/landing/agl_volume/` , with 6 subfolders mirroring your local folder names exactly
Medallion schemas	`workspace.bronze` , `workspace.silver` , `workspace.gold`
Ingestion mechanism	Auto Loader (`cloudFiles`), no database connector anywhere
Near-real-time detection	6 Databricks Jobs, one per domain folder, each on a **File arrival trigger**
Notebooks built in this guide	`00_setup` , `01_bronze_ingestion` , `02_silver_transformation` , `03_gold_layer`
CI/CD mechanism	Databricks Asset Bundles (DABs) + GitHub Actions

> **PREREQUISITE**
> A Databricks workspace with Unity Catalog enabled, a cluster or SQL warehouse you can attach notebooks to, and a GitHub account. The `spark-xml` library must be installed on your cluster (Compute → your cluster → Libraries → Install New → Maven → `com.databricks:spark-xml_2.12:0.18.0`) since 3 of the 31 sources are ingested as XML.

TABLE OF CONTENTS

Part 0 — Data Engineering Concepts You Need

Part 1 — Environment Setup: Catalog, Schemas, Volume

Part 2 — Manual Upload Workflow

Part 3 — Bronze Layer: Auto Loader for All 31 Tables, All 4 Formats

Part 4 — Near-Real-Time Detection: 6 File-Arrival-Triggered Jobs

Part 5 — Silver Layer: the 12 Industrial Transformations

Part 6 — Gold Layer: Business & ML Solutions

Part 7 — CI/CD: Databricks Asset Bundles + GitHub Actions

Part 8 — Troubleshooting Reference

Part 9 — Glossary

PART 0

Data Engineering Concepts You Need

0.1 The medallion architecture, and why three layers

> **CONCEPT**
> **Bronze** is raw data, landed as-is, with as little transformation as possible — its only job is to capture what arrived, reliably and exactly once, even if the source data is messy. **Silver** is where you clean, cast, deduplicate, join and standardize — this is "the truth, tidied up." **Gold** is shaped for a specific consumer (a dashboard, a model, a report) — pre-aggregated, denormalized, named in business language. The reason to keep them separate: if a transformation bug ships, you can rebuild Silver/Gold from Bronze without re-ingesting anything, because Bronze never throws data away.

0.2 Delta Lake

Every table in this guide is a Delta table (Databricks' default format). Delta gives you: ACID transactions (a failed write never leaves a half-written table), `MERGE INTO` for upserts, time travel (`SELECT * FROM t VERSION AS OF 3`), and schema enforcement/evolution. You don't choose this — it's the default when you write with `.saveAsTable()` or `.toTable()` on Databricks.

0.3 Auto Loader (`cloudFiles`)

> **CONCEPT**
> Auto Loader is a Spark Structured Streaming source built for exactly this use case: watch a cloud storage path (here, a Volume) and incrementally process only the files it hasn't seen. It tracks state in a **checkpoint** location (a hidden folder holding which files were processed and the inferred schema) — delete the checkpoint and it starts over; keep it and re-running the same notebook is always safe (idempotent), because already-processed files are simply skipped.

- **Schema inference & evolution** — on first run, Auto Loader samples the files and infers a schema, saved to `cloudFiles.schemaLocation`. Set `cloudFiles.schemaEvolutionMode = "addNewColumns"` so a source that later adds a column doesn't break the stream.
- **Triggers** — `trigger(availableNow=True)` processes everything currently sitting in the folder, then stops (a "batch" run of a streaming query — perfect for a Job). Omitting it makes the query run forever, polling for new files (useful only if you keep a cluster running 24/7).
- **File arrival triggers** (Part 4) are a different mechanism — a property of the *Databricks Job*, not the notebook. Databricks watches a Volume path at the platform level and starts a fresh job run within roughly a minute of a new file appearing, without any cluster running in between. This is what makes "upload a file → it gets processed automatically" cheap: no cluster idles waiting.

0.4 Unity Catalog hierarchy

`catalog.schema.table` is the three-level namespace for everything, and a **Volume** is Unity Catalog's managed way of exposing a folder of raw files (`/Volumes/catalog/schema/volume_name/…`) to notebooks and Auto Loader, with the same governance (grants, lineage) as a table.

0.5 Idempotency & checkpoints

Every ingestion and transformation cell in this guide is safe to re-run. Bronze is idempotent because of Auto Loader's checkpoint. Silver's `CREATE OR REPLACE TABLE` statements are idempotent because they always rebuild from Bronze rather than incrementally appending. This matters because a file-arrival-triggered job can, in rare cases, fire twice for the same file — the pipeline must not double-count if that happens.

0.6 The 12 industrial transformations, in one table

Part 5 implements all twelve of these against real columns from your dataset. Here's what each means before you see the code:

#	Transformation	One-line definition
1	Bucketing / Binning	Group a continuous numeric column into named ranges
2	Normalization / Standardization	Rescale a numeric column onto a common 0–1 or z-score range
3	Aggregation	Collapse many rows into a summary row via SUM/AVG/COUNT + GROUP BY
4	Joins and Merges	Combine two tables on a shared key
5	Pivoting / Unpivoting	Reshape rows into columns (wide) or columns into rows (long)
6	Deduplication	Keep exactly one row per logical key, deterministically
7	Filtering	Keep only rows that satisfy a business rule; route the rest to quarantine
8	Data Type Casting	Convert a column to the type it should have been (string→timestamp, etc.)
9	Window Functions	Compute a value relative to a partition/order without collapsing rows
10	Conditional Transformation	Derive a new column with CASE/WHEN business logic
11	Data Masking / Obfuscation	Irreversibly or reversibly hide PII while keeping the column usable
12	Schema Evolution	Accept new/changed source columns without breaking the pipeline

PART 1

Environment Setup: Catalog, Schemas, Volume

1.1 Notebook: 00_setup

```
catalog = "workspace"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.landing")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.bronze")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.silver")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.gold")

spark.sql(f"CREATE VOLUME IF NOT EXISTS {catalog}.landing.agl_volume")

# subfolders mirror your local folder names exactly - this is what you'll upload into in Part 2
domains = [
    "agl_data_output",
    "agl_fleet_data_output",
    "agl_driver_data_output",
    "agl_coldchain_data_output",
    "agl_delivery_data_output",
    "agl_external_data_output",
]
base = f"/Volumes/{catalog}/landing/agl_volume"
for d in domains:
    dbutils.fs.mkdirs(f"{base}/{d}")
dbutils.fs.mkdirs(f"{base}/_checkpoints")
dbutils.fs.mkdirs(f"{base}/_quarantine")

print("Schemas, volume and 6 domain folders ready.")
```

Expected result: `workspace.landing`, `workspace.bronze`, `workspace.silver`, `workspace.gold` *appear in Catalog Explorer, and* `/Volumes/workspace/landing/agl_volume/` *shows 6 domain subfolders plus* `_checkpoints` *and* `_quarantine`.

1.2 Cluster requirement: the spark-xml library

Three sources (`vehicles`, `warehouses`, `proof_of_delivery`) are ingested as XML, and Spark has no built-in XML reader. Install `com.databricks:spark-xml_2.12:0.18.0` from Maven onto whichever cluster you attach these notebooks to (Compute → cluster → Libraries tab → Install New). Without it, `spark.read.format("xml")` fails with `ClassNotFoundException`.

PART 2

Manual Upload Workflow

You upload manually — there is no automated pull from anywhere. Two ways to do it, both land files in the same place:

2.1 Option A — Catalog Explorer UI

1. Catalog → `workspace` → `landing` → Volumes → `agl_volume`.
2. Open the subfolder matching the domain you're uploading (e.g. `agl_fleet_data_output`).
3. Click **Upload to this volume** and drop in the specific format file(s) listed for that domain in Part 3.2 — not all 4 formats, just the one assigned per table.

2.2 Option B — Databricks CLI (faster for many files)

```
databricks fs cp "agl_fleet_data_output/vehicles.xml"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
databricks fs cp "agl_fleet_data_output/gps_tracking.parquet"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
databricks fs cp --recursive "agl_driver_data_output"
/Volumes/workspace/landing/agl_volume/agl_driver_data_output/ --overwrite
```

> **WATCH OUT**
> If you copy an entire local folder with `--recursive`, you'll upload all 4 formats for every table, and Auto Loader (configured in Part 3 to glob a specific extension per table) will simply ignore the formats it's not watching — harmless, but it wastes storage. Cleaner to copy only the files named in Part 3.2's format column.

2.3 Naming convention for future drops

When you generate a new batch later, upload it under a dated filename next to the original — e.g. `gps_tracking_20260801.parquet` next to `gps_tracking.parquet`. Every glob in Part 3 is `<table>*.<ext>`, so the new file is picked up automatically, and because it triggers a File Arrival job (Part 4), you don't even need to remember to re-run anything.

PART 3

Bronze Layer: Auto Loader for All 31 Tables, All 4 Formats

3.1 Design: one notebook, parameterized by domain

Rather than 31 hand-written cells, `01_bronze_ingestion` takes a `domain` widget parameter (one of the 6 folder names) and only ingests that domain's sources. This is what lets Part 4 wire up 6 independent File Arrival jobs — each job passes a different `domain` value to the same notebook.

```
dbutils.widgets.text("domain", "agl_fleet_data_output")
domain_param = dbutils.widgets.get("domain")

catalog = "workspace"
volume_path = "/Volumes/workspace/landing/agl_volume"
checkpoint_base = f"{volume_path}/_checkpoints"
bronze_schema = "bronze"
```

3.2 The full 31-source config, grouped by domain and format

Format was chosen per table to exercise all 4 code paths repeatedly, honoring what's actually available (the 3 master tables only ship as CSV/JSON — see the Data Dictionary's Part 7).

domain (folder)	table	format	bronze target
agl_data_output	agl_shipment_data	CSV	shipment_data
	customers_master	CSV	dim_customer
	shippers_master	JSON	dim_shipper
	consignees_master	CSV	dim_consignee
agl_fleet_data_output	vehicles	**XML**	dim_vehicle
	trips	CSV	fleet_trips
	gps_tracking	Parquet	fleet_gps_tracking
	telematics	Parquet	fleet_telematics
	maintenance_logs	CSV	fleet_maintenance_logs
agl_driver_data_output	drivers	CSV	dim_driver
	behavioral_events	JSON	driver_behavioral_events
	driver_logs	JSON	driver_logs
	app_telematics	CSV	driver_app_telematics
	fatigue_data	JSON	driver_fatigue_data
	pre_trip_inspections	CSV	driver_pre_trip_inspections
agl_coldchain_data_output	containers	CSV	dim_container
	warehouses	**XML**	dim_warehouse
	sensors	CSV	dim_sensor

	sensor_readings	Parquet	coldchain_sensor_readings
	reefer_status	Parquet	coldchain_reefer_status
	sensor_health	CSV	coldchain_sensor_health
agl_delivery_data_output	milestones	JSON	delivery_milestones
	exceptions	JSON	delivery_exceptions
	osd_logs	CSV	delivery_osd_logs
	proof_of_delivery	**XML**	delivery_proof_of_delivery
	delivery_metrics	CSV	delivery_metrics
agl_external_data_output	weather_data	JSON	ext_weather_data
	traffic_data	JSON	ext_traffic_data
	fuel_prices	CSV	ext_fuel_prices
	exchange_rates	CSV	ext_exchange_rates
	market_data	CSV	ext_market_data

```
bronze_sources = {
  "agl_data_output": [
    {"name": "agl_shipment_data",   "format": "csv",  "glob": "agl_shipment_data*.csv",   "target":
"shipment_data"},
    {"name": "customers_master",    "format": "csv",  "glob": "customers_master*.csv",    "target":
"dim_customer"},
    {"name": "shippers_master",     "format": "json", "glob": "shippers_master*.json",    "target":
"dim_shipper"},
    {"name": "consignees_master",   "format": "csv",  "glob": "consignees_master*.csv",   "target":
"dim_consignee"},
  ],
  "agl_fleet_data_output": [
    {"name": "trips",             "format": "csv",     "glob": "trips*.csv",             "target":
"fleet_trips"},
    {"name": "gps_tracking",      "format": "parquet", "glob": "gps_tracking*.parquet",  "target":
"fleet_gps_tracking"},
    {"name": "telematics",        "format": "parquet", "glob": "telematics*.parquet",    "target":
"fleet_telematics"},
    {"name": "maintenance_logs",  "format": "csv",     "glob": "maintenance_logs*.csv",  "target":
"fleet_maintenance_logs"},
    # vehicles.xml is handled separately in 3.4 (XML path)
  ],
  "agl_driver_data_output": [
    {"name": "drivers",              "format": "csv",  "glob": "drivers*.csv",              "target":
"dim_driver"},
    {"name": "behavioral_events",    "format": "json", "glob": "behavioral_events*.json",   "target":
"driver_behavioral_events"},
    {"name": "driver_logs",          "format": "json", "glob": "driver_logs*.json",         "target":
"driver_logs"},
    {"name": "app_telematics",       "format": "csv",  "glob": "app_telematics*.csv",       "target":
"driver_app_telematics"},
    {"name": "fatigue_data",         "format": "json", "glob": "fatigue_data*.json",        "target":
"driver_fatigue_data"},
    {"name": "pre_trip_inspections", "format": "csv",  "glob": "pre_trip_inspections*.csv", "target":
"driver_pre_trip_inspections"},
  ],
  "agl_coldchain_data_output": [
    {"name": "containers",       "format": "csv",     "glob": "containers*.csv",       "target":
"dim_container"},
    {"name": "sensors",          "format": "csv",     "glob": "sensors*.csv",          "target":
"dim_sensor"},
```

```
    {"name": "sensor_readings",  "format": "parquet", "glob": "sensor_readings*.parquet", "target":
"coldchain_sensor_readings"},
    {"name": "reefer_status",    "format": "parquet", "glob": "reefer_status*.parquet",   "target":
"coldchain_reefer_status"},
    {"name": "sensor_health",    "format": "csv",     "glob": "sensor_health*.csv",    "target":
"coldchain_sensor_health"},
    # warehouses.xml is handled separately in 3.4 (XML path)
  ],
  "agl_delivery_data_output": [
    {"name": "milestones",        "format": "json", "glob": "milestones*.json",       "target":
"delivery_milestones"},
    {"name": "exceptions",        "format": "json", "glob": "exceptions*.json",       "target":
"delivery_exceptions"},
    {"name": "osd_logs",          "format": "csv",  "glob": "osd_logs*.csv",          "target":
"delivery_osd_logs"},
    {"name": "delivery_metrics",  "format": "csv",  "glob": "delivery_metrics*.csv",  "target":
"delivery_metrics"},
    # proof_of_delivery.xml is handled separately in 3.4 (XML path)
  ],
  "agl_external_data_output": [
    {"name": "weather_data",     "format": "json", "glob": "weather_data*.json",     "target":
"ext_weather_data"},
    {"name": "traffic_data",     "format": "json", "glob": "traffic_data*.json",     "target":
"ext_traffic_data"},
    {"name": "fuel_prices",      "format": "csv",  "glob": "fuel_prices*.csv",       "target":
"ext_fuel_prices"},
    {"name": "exchange_rates",   "format": "csv",  "glob": "exchange_rates*.csv",    "target":
"ext_exchange_rates"},
    {"name": "market_data",      "format": "csv",  "glob": "market_data*.csv",       "target":
"ext_market_data"},
  ],
}
```

3.3 Generic ingestion function — handles CSV, JSON and Parquet

```
def ingest_source(domain_folder, src):
    reader = (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", src["format"])
        .option("cloudFiles.schemaLocation", f"{checkpoint_base}/{src['target']}_schema")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    )
    if src["format"] == "csv":
        reader = reader.option("header", "true")
    if src["format"] == "json":
        reader = reader.option("multiLine", "true")

    df = reader.load(f"{volume_path}/{domain_folder}/{src['glob']}")

    (df.writeStream
       .format("delta")
       .option("checkpointLocation", f"{checkpoint_base}/{src['target']}_ckpt")
       .option("mergeSchema", "true")
       .outputMode("append")
       .trigger(availableNow=True)
       .toTable(f"{catalog}.{bronze_schema}.{src['target']}")
       .awaitTermination())

results = []
for src in bronze_sources.get(domain_param, []):
    try:
        ingest_source(domain_param, src)
        results.append((src["target"], "OK"))
        print(f"{src['target']}: batch complete.")
```

```
    except Exception as e:
        results.append((src["target"], f"FAILED: {e}"))
        print(f"{src['target']}: FAILED — {e}")

print(f"\n{domain_param}: {sum(1 for r in results if r[1]=='OK')}/{len(results)} sources ingested.")
```

Expected result: when `domain` = `agl_fleet_data_output`, 4 "batch complete" lines print (trips, gps_tracking, telematics, maintenance_logs) — `vehicles` is deliberately excluded here because it's XML.

3.4 The XML path — generalized manifest-table fallback

Auto Loader's `cloudFiles` connector does not support XML, so these 3 sources use a manifest table that remembers which files were already read, and re-reads only the new ones each run — safe to re-run, same idempotency guarantee as Auto Loader's checkpoint, just implemented by hand.

```
from pyspark.sql.functions import col
from pyspark.sql import Row
from datetime import datetime

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.landing._processed_xml_files (
    file_path STRING, target_table STRING, processed_at TIMESTAMP
)""")

def ingest_xml_source(domain_folder, glob_prefix, row_tag, target_table, column_map):
    all_files = [f.path for f in dbutils.fs.ls(f"{volume_path}/{domain_folder}")
                 if f.name.startswith(glob_prefix) and f.name.endswith(".xml")]
    already = set(r["file_path"] for r in spark.sql(f"""
        SELECT file_path FROM {catalog}.landing._processed_xml_files WHERE target_table = '{target_table}'
    """).collect())
    new_files = [f for f in all_files if f not in already]

    if not new_files:
        print(f"{target_table}: no new XML files.")
        return

    df_raw = spark.read.format("xml").option("rowTag", row_tag).load(new_files)
    df_clean = df_raw.select(*[col(src).alias(dst) for src, dst in column_map.items()])
    df_clean.write.mode("append").option("mergeSchema", "true").saveAsTable(f"{catalog}.{bronze_schema}.
{target_table}")

    spark.createDataFrame(
        [Row(file_path=f, target_table=target_table, processed_at=datetime.utcnow()) for f in new_files]
    ).write.mode("append").saveAsTable(f"{catalog}.landing._processed_xml_files")
    print(f"{target_table}: processed {len(new_files)} new XML file(s).")

# --- vehicles.xml : root , row tag , flat lowercase-no-underscore tags ---
vehicle_column_map = {
    "vehicleid": "vehicle_id", "licenseplate": "license_plate", "model": "model", "year": "year",
    "fuelcapacityl": "fuel_capacity_l", "avgconsumptionlper100km": "avg_consumption_l_per_100km",
    "currentfuellevel": "current_fuel_level", "odometerreadingkm": "odometer_reading_km",
    "trucktype": "truck_type", "maxloadtonnes": "max_load_tonnes", "status": "status",
    "assigneddriver": "assigned_driver", "assignedcorridor": "assigned_corridor",
    "insuranceexpiry": "insurance_expiry", "lastinspectiondate": "last_inspection_date",
    "nextinspectiondate": "next_inspection_date",
}
if domain_param == "agl_fleet_data_output":
    ingest_xml_source("agl_fleet_data_output", "vehicles", "Vehicle", "dim_vehicle", vehicle_column_map)

# --- warehouses.xml : root , row tag  ---
warehouse_column_map = {
    "warehouseid": "warehouse_id", "warehousename": "warehouse_name", "address": "address",
    "totalaream2": "total_area_m2", "coolingcapacitykw": "cooling_capacity_kw",
```

```
    "numberofbays": "number_of_bays", "producttypes": "product_types", "temperaturezone":
"temperature_zone",
    "powerbackuphours": "power_backup_hours", "generatorcapacitykva": "generator_capacity_kva",
    "managername": "manager_name", "contactphone": "contact_phone", "contactemail": "contact_email",
    "operatinghours": "operating_hours", "certification": "certification", "status": "status",
}
if domain_param == "agl_coldchain_data_output":
    ingest_xml_source("agl_coldchain_data_output", "warehouses", "Warehouse", "dim_warehouse",
warehouse_column_map)

# --- proof_of_delivery.xml : root , row tag  (not "ProofOfDelivery" - verify before trusting) ---
pod_column_map = {
    "podid": "pod_id", "shipmentid": "shipment_id", "recipientname": "recipient_name",
    "recipientcontact": "recipient_contact", "recipientemail": "recipient_email",
    "recipientsignature": "recipient_signature", "deliveryagentname": "delivery_agent_name",
    "deliveryagentsignature": "delivery_agent_signature", "deliverytimestamp": "delivery_timestamp",
    "deliverylocation": "delivery_location", "cargoconditionsummary": "cargo_condition_summary",
    "cargoconditiondetails": "cargo_condition_details", "photoevidence": "photo_evidence",
    "temperatureatdeliveryc": "temperature_at_delivery_c", "humidityatdeliverypercent":
"humidity_at_delivery_percent",
    "deliverynotes": "delivery_notes", "receivedby": "received_by",
}
if domain_param == "agl_delivery_data_output":
    ingest_xml_source("agl_delivery_data_output", "proof_of_delivery", "POD", "delivery_proof_of_delivery",
pod_column_map)
```

WATCH OUT

The row tag for `proof_of_delivery.xml` is `<POD>`, not `<ProofOfDelivery>` — the root element and the row element are different names. Always open the actual XML file and check both before wiring up `rowTag`; guessing it from the table name is exactly the mistake to avoid (verified directly against the file for this guide).

NOTE

`warehouses.xml` has a real encoding defect: `temperature_zone` contains `"Ambient (15Â°C to 25Â°C)"` — a mojibake double-encoding of the ° symbol. Bronze should land it as-is (raw is raw); fix it in Silver with `regexp_replace(col, "Â°", "°")` (Part 5.1).

3.5 Special ingestion notes for 3 more sources

Table	Issue	Bronze policy	Fixed in
app_telematics.last_known_location	Arrives as a Python-dict-repr string with single quotes — `"{'lat': -1.890936, 'lon': 29.909521}"` — **not valid JSON** (double quotes required for `from_json`)	Land as raw string, untouched	Silver 5.1 (Data Type Casting)
pre_trip_inspections.inspection_items	A proper embedded JSON array string	Land as raw string	Silver 5.1
proof_of_delivery.cargo_condition_details / photo_evidence	Proper embedded JSON array strings (XML-entity-escaped in the source, auto-unescaped by the XML reader)	Land as raw string	Silver 5.1

PART 4

Near-Real-Time Detection: 6 File-Arrival-Triggered Jobs

4.1 Why 6 jobs and not 1

A Databricks Job can only have **one** trigger. Since you want "upload a file into any of the 6 folders → that folder's data gets processed automatically," the correct shape is **one Job per folder**, each watching its own path, each running `01_bronze_ingestion` with a different `domain` parameter. They're fully independent — uploading into `agl_driver_data_output` never triggers the fleet job.

4.2 Create the 6 jobs

Repeat these steps 6 times, once per domain folder:

1. Workflows → **Create Job**. Name it e.g. `AGL_Bronze_Fleet`.
2. Task: name `ingest`, type Notebook, path `01_bronze_ingestion`.
3. Under the task's **Parameters**, add base parameter `domain` = `agl_fleet_data_output` (matching the folder name exactly).
4. Compute: a small Job Cluster is enough — these are lightweight batch runs.
5. In **Schedules & Triggers**, click **Add trigger**, set Trigger type to **File arrival**, and point it at `/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/`.
6. Save.

Job name	domain parameter / watched path
AGL_Bronze_Shipment	agl_data_output
AGL_Bronze_Fleet	agl_fleet_data_output
AGL_Bronze_Driver	agl_driver_data_output
AGL_Bronze_Coldchain	agl_coldchain_data_output
AGL_Bronze_Delivery	agl_delivery_data_output
AGL_Bronze_External	agl_external_data_output

Expected result: upload a new file into, say, `agl_coldchain_data_output/` — within about a minute, `AGL_Bronze_Coldchain` starts a run automatically, ingests just that domain's sources, and stops. No cluster runs in between uploads.

> **TIP**
> If a minute of latency isn't fast enough, swap the trigger for a continuously running streaming notebook (drop `trigger(availableNow=True)` and instead use `trigger(processingTime="10 seconds")`, then run the notebook itself, not as a Job, on an always-on cluster). That gets you near-instant pickup but you pay for the cluster 24/7 — the file-arrival design above is the standard cost/latency trade Databricks recommends for this exact scenario.

PART 5

Silver Layer: the 12 Industrial Transformations

Notebook: `02_silver_transformation`. Each subsection below is one of your 12 requested transformation types, applied to real columns — not toy examples.

5.1 Data Type Casting

```
from pyspark.sql.functions import regexp_replace, from_json, col
from pyspark.sql.types import StructType, StructField, DoubleType, ArrayType, StringType

# fix the warehouses.xml mojibake, then parse the two embedded-JSON string columns per Part 3.5
warehouse_fixed = (
    spark.table("workspace.bronze.dim_warehouse")
    .withColumn("temperature_zone", regexp_replace(col("temperature_zone"), "Â°", "°"))
)
warehouse_fixed.write.mode("overwrite").saveAsTable("workspace.silver.dim_warehouse")

loc_schema = StructType([StructField("lat", DoubleType()), StructField("lon", DoubleType())])
app_telematics_fixed = (
    spark.table("workspace.bronze.driver_app_telematics")
    # Python-dict-repr -> valid JSON: single quotes to double quotes
    .withColumn("location_json", regexp_replace(col("last_known_location"), "'", '"'))
    .withColumn("location_parsed", from_json(col("location_json"), loc_schema))
    .withColumn("last_known_lat", col("location_parsed.lat"))
    .withColumn("last_known_lon", col("location_parsed.lon"))
    .drop("last_known_location", "location_json", "location_parsed")
)
app_telematics_fixed.write.mode("overwrite").saveAsTable("workspace.silver.driver_app_telematics")

items_schema = ArrayType(StructType([
    StructField("item", StringType()), StructField("status", StringType()), StructField("notes",
StringType())
]))
pti_fixed = (
    spark.table("workspace.bronze.driver_pre_trip_inspections")
    .withColumn("inspection_items_parsed", from_json(col("inspection_items"), items_schema))
)
pti_fixed.write.mode("overwrite").saveAsTable("workspace.silver.driver_pre_trip_inspections")
```

Expected result: `silver.driver_app_telematics` has real `last_known_lat` / `last_known_lon` double columns instead of a string; `silver.dim_warehouse.temperature_zone` reads correctly ("15°C to 25°C").

5.2 Deduplication

Industrial practice: don't just `dropDuplicates()` — decide which duplicate survives (the latest by a timestamp column) using a window function, so a re-ingested/updated row wins deterministically.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, desc

w = Window.partitionBy("shipment_id").orderBy(desc("updated_at"))
shipment_dedup = (
    spark.table("workspace.bronze.shipment_data")
    .withColumn("rn", row_number().over(w))
```

```
    .filter("rn = 1")
    .drop("rn")
)
shipment_dedup.write.mode("overwrite").saveAsTable("workspace.silver.shipment_data")
```

5.3 Filtering

```
from pyspark.sql.functions import lit

bad_rows = shipment_dedup.filter("weight_kg <= 0 OR volume_m3 <= 0 OR piece_count <= 0")
good_rows = shipment_dedup.filter("weight_kg > 0 AND volume_m3 > 0 AND piece_count > 0")

bad_rows.withColumn("quarantine_reason", lit("non-positive weight/volume/piece_count")) \
    .write.mode("append").saveAsTable("workspace.silver.shipment_data_quarantine")
good_rows.write.mode("overwrite").saveAsTable("workspace.silver.shipment_data")
print(f"Quarantined {bad_rows.count()} of {shipment_dedup.count()} shipment rows.")
```

5.4 Conditional Transformation

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.dim_customer AS
SELECT *,
    CASE
        WHEN credit_limit >= 75000 THEN 'High Value'
        WHEN credit_limit >= 30000 THEN 'Standard'
        ELSE 'Low Value'
    END AS customer_value_tier
FROM workspace.bronze.dim_customer
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.shipment_data AS
SELECT *,
    CASE
        WHEN actual_delivery_date IS NULL AND delivery_deadline < current_timestamp() THEN 'Overdue - In
Transit'
        WHEN actual_delivery_date > delivery_deadline THEN 'Delivered Late'
        WHEN actual_delivery_date IS NOT NULL THEN 'Delivered On Time'
        ELSE 'In Transit - On Schedule'
    END AS delivery_status_flag
FROM workspace.silver.shipment_data
""")
```

5.5 Bucketing / Binning

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.dim_driver AS
SELECT *,
    CASE
        WHEN total_experience_years < 2  THEN 'Junior (0-2y)'
        WHEN total_experience_years < 5  THEN 'Mid (2-5y)'
        WHEN total_experience_years < 10 THEN 'Senior (5-10y)'
        ELSE 'Veteran (10y+)'
    END AS experience_band
FROM workspace.bronze.dim_driver
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.dim_vehicle AS
SELECT *,
```

```
    (year(current_date()) - CAST(year AS INT)) AS vehicle_age_years,
    CASE
        WHEN (year(current_date()) - CAST(year AS INT)) <= 3  THEN 'New (0-3y)'
        WHEN (year(current_date()) - CAST(year AS INT)) <= 7  THEN 'Mid-life (4-7y)'
        ELSE 'Aging (8y+)'
    END AS vehicle_age_band
FROM workspace.bronze.dim_vehicle
""")
```

5.6 Normalization / Standardization

```
from pyspark.sql.functions import mean as fmean, stddev, min as fmin, max as fmax

stats = spark.table("workspace.silver.driver_behavioral_events") \
    .agg(fmean("speed_kmh").alias("mu"), stddev("speed_kmh").alias("sigma"),
         fmin("speed_kmh").alias("lo"), fmax("speed_kmh").alias("hi")).collect()[0]

behavioral_scaled = spark.table("workspace.silver.driver_behavioral_events").withColumn(
    "speed_kmh_zscore", (col("speed_kmh") - stats["mu"]) / stats["sigma"]
).withColumn(
    "speed_kmh_minmax", (col("speed_kmh") - stats["lo"]) / (stats["hi"] - stats["lo"])
)
behavioral_scaled.write.mode("overwrite").saveAsTable("workspace.silver.driver_behavioral_events")
```

Z-score flags outlier speeds for anomaly detection; min-max gives ML models a bounded 0–1 feature — both derived from the same column for different downstream consumers.

5.7 Window Functions

```
from pyspark.sql.functions import rank, avg
from pyspark.sql.window import Window

# 1. rank drivers by rating within their home terminal
w_rank = Window.partitionBy("home_terminal").orderBy(desc("rating"))
driver_ranked = spark.table("workspace.silver.dim_driver").withColumn("rating_rank_in_terminal",
rank().over(w_rank))
driver_ranked.write.mode("overwrite").saveAsTable("workspace.silver.dim_driver")

# 2. latest sensor reading per sensor (row_number, not dedup - keep full history elsewhere)
w_latest = Window.partitionBy("sensor_id").orderBy(desc("timestamp"))
latest_readings = (
    spark.table("workspace.bronze.coldchain_sensor_readings")
    .withColumn("rn", row_number().over(w_latest))
    .filter("rn = 1").drop("rn")
)
latest_readings.write.mode("overwrite").saveAsTable("workspace.silver.coldchain_latest_reading_per_sensor")

# 3. 7-day rolling average temperature per corridor
w_roll = Window.partitionBy("corridor_name").orderBy("event_date").rowsBetween(-6, 0)
weather_with_date = spark.table("workspace.bronze.ext_weather_data").withColumn("event_date",
col("timestamp").cast("date"))
weather_rolling = weather_with_date.withColumn("temp_c_7day_avg", avg("temperature_c").over(w_roll))
weather_rolling.write.mode("overwrite").saveAsTable("workspace.silver.ext_weather_data")
```

5.8 Aggregation

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.fleet_maintenance_cost_by_vehicle AS
SELECT vehicle_id,
       COUNT(*) AS service_count,
       SUM(cost) AS total_maintenance_cost,
```

```
       AVG(cost) AS avg_maintenance_cost,
       MAX(cost) AS max_single_cost,
       SUM(CASE WHEN breakdown_recorded THEN 1 ELSE 0 END) AS breakdown_count
FROM workspace.bronze.fleet_maintenance_logs
GROUP BY vehicle_id
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.ext_corridor_conditions_daily AS
SELECT corridor_name, CAST(timestamp AS DATE) AS event_date,
       AVG(temperature_c) AS avg_temp_c, MAX(precipitation_mm) AS max_precip_mm,
       AVG(humidity_percent) AS avg_humidity_pct
FROM workspace.bronze.ext_weather_data
GROUP BY corridor_name, CAST(timestamp AS DATE)
""")
```

5.9 Joins and Merges

The clean joins first — these use keys verified to match in the Data Dictionary's Part 8:

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.fleet_trip_enriched AS
SELECT t.*, v.truck_type, v.max_load_tonnes, v.vehicle_age_band, v.status AS vehicle_status
FROM workspace.silver.fleet_trips t
LEFT JOIN workspace.silver.dim_vehicle v ON t.vehicle_id = v.vehicle_id
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.coldchain_reading_enriched AS
SELECT r.*, c.product_type, c.setpoint_temp_c, c.temp_min_c, c.temp_max_c,
       CASE WHEN r.temperature_c < c.temp_min_c OR r.temperature_c > c.temp_max_c THEN 1 ELSE 0 END AS
temp_excursion_flag
FROM workspace.bronze.coldchain_sensor_readings r
LEFT JOIN workspace.silver.dim_container c ON r.container_id = c.container_id
""")
```

> **WATCH OUT — THE TWO MISMATCHED JOINS**
> Per the Data Dictionary Part 8, `fleet.vehicle_id` (`VEH_000001`, 6-digit) cannot be joined directly to `driver domain.vehicle_id` (`VEH_0001`, 4-digit), and `agl_shipment_data.shipment_id` (`SHIP_########`) cannot be joined directly to the delivery domain's `shipment_id` (`AGL-########`). Both sides have identical row counts (50 vehicles, 1,000 shipments), which makes a **positional crosswalk** tempting — but that only works if both tables were generated in the same order, which cannot be confirmed from the data alone. Below is the pattern for building one, clearly labeled as a heuristic to validate, not a verified join:

```
# HEURISTIC BRIDGE - validate against a real source system before trusting this in production
from pyspark.sql.functions import row_number
from pyspark.sql.window import Window

fleet_ordered  = spark.table("workspace.silver.dim_vehicle").withColumn("seq",
row_number().over(Window.orderBy("vehicle_id")))
driver_vehicle_ids =
spark.table("workspace.bronze.driver_behavioral_events").select("vehicle_id").distinct() \
    .withColumn("seq", row_number().over(Window.orderBy("vehicle_id")))

vehicle_id_bridge = fleet_ordered.select("vehicle_id", "seq") \
    .join(driver_vehicle_ids.withColumnRenamed("vehicle_id", "driver_domain_vehicle_id"), "seq") \
    .select("vehicle_id", "driver_domain_vehicle_id")
vehicle_id_bridge.write.mode("overwrite").saveAsTable("workspace.silver._bridge_vehicle_id_heuristic")
print("Bridge built - review workspace.silver._bridge_vehicle_id_heuristic manually before using it
downstream.")
```

5.10 Pivoting / Unpivoting

Pivot — `milestones` is 10 rows per shipment; turn it into one row per shipment with a timestamp column per milestone, a natural operations-dashboard shape:

```
milestone_wide = (
    spark.table("workspace.bronze.delivery_milestones")
    .groupBy("shipment_id")
    .pivot("milestone_name", [
        "Booking Confirmed", "Cargo Received", "Departed Origin", "Arrived at Border",
        "Customs Cleared", "Border Cleared", "Departed Border", "Arrived at Destination",
        "Out for Delivery", "Delivered",
    ])
    .agg({"timestamp": "max"})
)
milestone_wide.write.mode("overwrite").saveAsTable("workspace.silver.delivery_milestones_wide")
```

Unpivot — `behavioral_events` has 12 sparse, event-type-specific columns (Part 9 of the Data Dictionary). Melt them into a tall attribute/value table so downstream consumers don't need to know all 12 column names in advance:

```
sparse_cols = [
    "deceleration_rate", "distance_to_obstacle_m", "acceleration_rate", "engine_load_percent",
    "speed_limit_kmh", "excess_speed_kmh", "idle_duration_minutes", "fuel_consumed_l",
    "duration_unbuckled_seconds", "warning_count", "lateral_acceleration", "curve_radius_m",
]
stack_expr = "stack({0}, {1}) as (attribute, value)".format(
    len(sparse_cols), ", ".join([f"'{c}', {c}" for c in sparse_cols])
)
behavioral_long = (
    spark.table("workspace.bronze.driver_behavioral_events")
    .selectExpr("event_id", "driver_id", "event_type", "timestamp", stack_expr)
    .filter("value IS NOT NULL")
)
behavioral_long.write.mode("overwrite").saveAsTable("workspace.silver.driver_behavioral_events_attributes_lo
ng")
```

5.11 Data Masking / Obfuscation

```
from pyspark.sql.functions import sha2, concat, regexp_extract, lit

def mask_email(colname):
    return concat(regexp_extract(col(colname), r"^(.)", 1), lit("***@"), regexp_extract(col(colname),
r"@(.*)$", 1))

def mask_phone(colname):
    return concat(lit("***-***-"), col(colname).substr(-4, 4))

dim_driver_masked = spark.table("workspace.silver.dim_driver") \
    .withColumn("email_masked", mask_email("email")) \
    .withColumn("phone_masked", mask_phone("phone")) \
    .withColumn("license_number_hash", sha2(col("license_number"), 256)) \
    .drop("email", "phone", "license_number", "emergency_contact", "emergency_phone")
dim_driver_masked.write.mode("overwrite").saveAsTable("workspace.silver.dim_driver_masked")

# signature blobs carry no analytical value and are PII-adjacent - drop from analytics-facing Silver
entirely
pod_analytics_safe = spark.table("workspace.silver.delivery_proof_of_delivery") \
```

```
    .drop("recipient_signature", "delivery_agent_signature", "recipient_email", "recipient_contact")
pod_analytics_safe.write.mode("overwrite").saveAsTable("workspace.silver.delivery_proof_of_delivery_safe")
```

> **NOTE**
> Keep the un-masked `dim_driver` table restricted (Unity Catalog row/column-level security or a separate, tightly-permissioned schema) and grant broad access only to `dim_driver_masked` — this is the standard GDPR/HIPAA-style pattern: one governed source of truth, one safe-to-share derived copy.

5.12 Schema Evolution

Ingestion-side evolution is already on (`cloudFiles.schemaEvolutionMode = "addNewColumns"`, Part 3.3). Silver needs the same tolerance on its own writes:

```
# mergeSchema lets a rebuilt Silver table pick up a brand-new Bronze column without failing
spark.table("workspace.bronze.shipment_data").write.mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("workspace.silver.shipment_data")

# for incremental (MERGE-based) Silver tables instead of full rebuilds, enable auto-merge:
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

`overwriteSchema` is for full `CREATE OR REPLACE` rebuilds (used throughout this guide); `autoMerge.enabled` is for tables you upsert into with `MERGE INTO` instead.

PART 6

Gold Layer: Business & ML Solutions

Notebook: `03_gold_layer`. Built only from the joins verified clean in Part 5.9 — no heuristic bridges in Gold without an explicit sign-off, since Gold is what analysts and models take at face value.

6.1 For Data Analysts — gold.vw_fleet_utilization

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold.vw_fleet_utilization AS
SELECT vehicle_id, vehicle_age_band, truck_type,
       COUNT(*) AS trip_count,
       SUM(actual_distance_km) AS total_km,
       AVG(actual_duration_hours - planned_duration_hours) AS avg_overrun_hours,
       AVG(idle_time_minutes) AS avg_idle_minutes
FROM workspace.silver.fleet_trip_enriched
GROUP BY vehicle_id, vehicle_age_band, truck_type
ORDER BY avg_idle_minutes DESC
""")
```

6.2 For Data Analysts — gold.vw_shipment_sla_performance

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold.vw_shipment_sla_performance AS
SELECT customer_value_tier, delivery_status_flag, currency,
       COUNT(*) AS shipment_count, SUM(total_cost) AS total_revenue,
       AVG(penalty_rate) AS avg_penalty_rate
FROM workspace.silver.shipment_data s
JOIN workspace.silver.dim_customer c ON s.customer_id = c.customer_id
GROUP BY customer_value_tier, delivery_status_flag, currency
""")
```

6.3 For Data Scientists — gold.fact_coldchain_spoilage_risk

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold.fact_coldchain_spoilage_risk AS
SELECT container_id, product_type, CAST(timestamp AS DATE) AS event_date,
       COUNT(*) AS reading_count,
       SUM(temp_excursion_flag) AS temp_excursion_count,
       AVG(temperature_c) AS avg_temp_c, MIN(temperature_c) AS min_temp_c, MAX(temperature_c) AS max_temp_c,
       CASE
           WHEN SUM(temp_excursion_flag) > 5 THEN 'High'
           WHEN SUM(temp_excursion_flag) BETWEEN 1 AND 5 THEN 'Medium'
           ELSE 'Low'
       END AS spoilage_risk_label
FROM workspace.silver.coldchain_reading_enriched
GROUP BY container_id, product_type, CAST(timestamp AS DATE)
""")
```

Feature-ready: one row per container-day, numeric features plus a label column, straight into `spark.table(...).toPandas()` for a classifier.

6.4 For Data Scientists — gold.fact_driver_safety_scorecard

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold.fact_driver_safety_scorecard AS
SELECT d.driver_id, d.experience_band, d.rating, d.rating_rank_in_terminal,
       COUNT(b.event_id) AS behavioral_event_count,
       SUM(CASE WHEN b.severity = 'Critical' THEN 1 ELSE 0 END) AS critical_event_count,
       SUM(CASE WHEN b.event_type = 'Harsh Braking' THEN 1 ELSE 0 END) AS harsh_braking_count
FROM workspace.silver.dim_driver d
LEFT JOIN workspace.silver.driver_behavioral_events b ON d.driver_id = b.driver_id
GROUP BY d.driver_id, d.experience_band, d.rating, d.rating_rank_in_terminal
""")
```

6.5 For ML Engineers — registering Gold tables as Feature Tables

```
from databricks.feature_engineering import FeatureEngineeringClient

fe = FeatureEngineeringClient()
fe.create_table(
    name="workspace.gold.fact_coldchain_spoilage_risk",
    primary_keys=["container_id", "event_date"],
    timestamp_keys=["event_date"],
    df=spark.table("workspace.gold.fact_coldchain_spoilage_risk"),
    description="Container-day cold-chain features for spoilage risk models.",
)
```

Registering through the Feature Engineering client (Unity Catalog-backed) gives you point-in-time correct joins for training and the same table for online/batch scoring — the standard way to avoid train/serve skew on Databricks.

PART 7

CI/CD: Databricks Asset Bundles + GitHub Actions

7.1 Concept: what a Bundle is

CONCEPT

A Databricks Asset Bundle (DAB) is a YAML description (`databricks.yml`) of everything this project needs on Databricks — notebooks, jobs, their triggers and parameters — checked into the same Git repo as your code. `databricks bundle deploy` reads that YAML and creates/updates the real jobs in your workspace to match it. This turns "click through the Workflows UI 6 times" (Part 4) into a file you can review in a pull request.

7.2 Repo layout

```
agl-lakehouse/
├── databricks.yml
├── notebooks/
│   ├── 00_setup.py
│   ├── 01_bronze_ingestion.py
│   ├── 02_silver_transformation.py
│   └── 03_gold_layer.py
└── .github/
    └── workflows/
        └── deploy.yml
```

7.3 databricks.yml — the 6 ingestion jobs + 1 curation job, as code

```
bundle:
  name: agl_lakehouse

variables:
  catalog:
    default: workspace
  volume_root:
    default: /Volumes/workspace/landing/agl_volume

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: https://.cloud.databricks.com
  prod:
    mode: production
    workspace:
      host: https://.cloud.databricks.com

resources:
  jobs:
    agl_bronze_shipment:
      name: "AGL Bronze - Shipment"
      tasks:
        - task_key: ingest
          notebook_task:
            notebook_path: ../notebooks/01_bronze_ingestion.py
```

```
          base_parameters: { domain: agl_data_output }
        new_cluster: &small_job_cluster
          spark_version: "15.4.x-scala2.12"
          node_type_id: "Standard_DS3_v2"
          num_workers: 1
    trigger:
      file_arrival:
        url: "${var.volume_root}/agl_data_output/"

  agl_bronze_fleet:
    name: "AGL Bronze - Fleet"
    tasks:
      - task_key: ingest
        notebook_task:
          notebook_path: ../notebooks/01_bronze_ingestion.py
          base_parameters: { domain: agl_fleet_data_output }
        new_cluster: *small_job_cluster
    trigger:
      file_arrival:
        url: "${var.volume_root}/agl_fleet_data_output/"

  # ... repeat the same shape for agl_bronze_driver, agl_bronze_coldchain,
  #     agl_bronze_delivery, agl_bronze_external (domain + url change only) ...

  agl_silver_gold_curation:
    name: "AGL Silver + Gold Curation"
    schedule:
      quartz_cron_expression: "0 0 */6 * * ?"
      timezone_id: "Africa/Kigali"
    tasks:
      - task_key: silver
        notebook_task: { notebook_path: ../notebooks/02_silver_transformation.py }
        new_cluster: *small_job_cluster
      - task_key: gold
        depends_on: [{ task_key: silver }]
        notebook_task: { notebook_path: ../notebooks/03_gold_layer.py }
        new_cluster: *small_job_cluster
```

> **NOTE**
> Silver/Gold stay on a schedule, not a file-arrival trigger — they need multiple domains' Bronze tables to be consistent at once, which a single-folder trigger can't guarantee. This mirrors the reasoning in Part 4.1.

7.4 Local validation before pushing

```
pip install databricks-cli  # or: brew install databricks/tap/databricks
databricks auth login --host https://.cloud.databricks.com

databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run agl_bronze_fleet -t dev   # smoke-test one job manually
```

7.5 GitHub Actions: validate on PR, deploy on merge

```
# .github/workflows/deploy.yml
name: Deploy AGL Lakehouse

on:
  pull_request:
    branches: [main]
  push:
    branches: [main, develop]
```

```
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - name: Validate bundle
        run: databricks bundle validate -t dev
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_DEV }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_DEV }}

  deploy_dev:
    if: github.ref == 'refs/heads/develop'
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle deploy -t dev
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_DEV }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_DEV }}

  deploy_prod:
    if: github.ref == 'refs/heads/main'
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST_PROD }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN_PROD }}
```

7.6 Wiring up the GitHub ↔ Databricks connection

1. **Create a Databricks Personal Access Token** (or, better for production, a service principal + OAuth token) in each workspace: User Settings → Developer → Access tokens → Generate new token.
2. **Store it in GitHub**: repo → Settings → Secrets and variables → Actions → New repository secret. Add `DATABRICKS_HOST_DEV`, `DATABRICKS_TOKEN_DEV`, `DATABRICKS_HOST_PROD`, `DATABRICKS_TOKEN_PROD`.
3. **Push the repo to GitHub** and open a PR from a feature branch into `develop` — the `validate` job runs automatically and comments/fails on any bundle error.
4. **Merge to `develop`** → `deploy_dev` runs, updates all 7 jobs in your dev workspace to match the YAML.
5. **Merge `develop` into `main`** → `deploy_prod` runs against the production workspace.

> **TIP**
> You can also link the repo directly inside Databricks (Workspace → Repos → Add Repo → paste the GitHub URL) for interactive development — pull latest, edit notebooks in the Databricks UI, commit and push from there. That's independent of the CI/CD pipeline above and mainly useful while you're actively writing notebooks; the GitHub Actions flow is what actually keeps your workspace's jobs in sync with what's merged.

PART 8

Troubleshooting Reference

Symptom	Fix
`spark.read.format("xml")` throws `ClassNotFoundException`	The spark-xml Maven library (Part 1.2) isn't installed on the cluster running the job.
A File Arrival job never fires	Confirm you uploaded straight into the domain's own subfolder — a job's trigger only watches the exact path it's configured with.
`from_json` on `last_known_location` returns all-null	You skipped the single-quote → double-quote fix in Part 5.1 — the source string isn't valid JSON as-is.
A join in Part 5.9 returns mostly-null enriched columns	Check whether it's one of the two confirmed-mismatched keys (vehicle_id 4- vs 6-digit, shipment_id SHIP_ vs AGL-) from the Data Dictionary Part 8 before assuming a code bug.
`databricks bundle deploy` fails with an auth error	Re-check the GitHub secret names match exactly what the workflow YAML references, and that the PAT hasn't expired.
Bronze ingestion re-processes a file you already ingested	Should not happen — if it does, check you didn't delete/move the `_checkpoints` folder, which is what Auto Loader uses to remember what it's already read.

PART 9

Glossary

Term	Meaning
Auto Loader / cloudFiles	Databricks' incremental file-ingestion streaming source; tracks processed files in a checkpoint.
File arrival trigger	A Job-level trigger that starts a run within ~1 minute of a new file appearing in a Volume path, with no cluster running in between.
Checkpoint	Hidden folder recording exactly which files/offsets a streaming job has already processed.
Schema evolution mode	Auto Loader setting controlling behavior when a new column appears in the source (`addNewColumns` accepts it).
Manifest-table fallback	A hand-rolled "already processed" tracking table used for formats (like XML) Auto Loader doesn't natively support.
Window function	A SQL/DataFrame function computed per-row relative to a partition and order, without collapsing rows (unlike GROUP BY).
Heuristic bridge/crosswalk table	A best-effort key mapping built when two tables' natural keys don't match — must be validated before being trusted in Gold.
Databricks Asset Bundle (DAB)	A YAML-defined, version-controlled description of the jobs/notebooks a project needs; deployed via the Databricks CLI.
Feature Engineering Client	The Unity Catalog-backed API for registering Gold tables as reusable, point-in-time-correct ML feature tables.

AGL Lakehouse — Medallion Implementation & CI/CD Guide. Built directly on the verified findings in AGL_Data_Dictionary_Verified.pdf.